



浙江大学爱丁堡大学联合学院

ZJU-UoE Institute

Python IV: Functions and Algorithms

Algorithm Complexity

IBI 1, Lecture 8.2

Dr Ying CHI – yingchi@intl.zju.edu.cn

Adapted from Dr Rob Young

Semester 2, 2025/26

Today we are going to learn

- What is an algorithm?
- How do we measure the growth of algorithms?
- Big "O" notation
- Bubble sort and merge sort

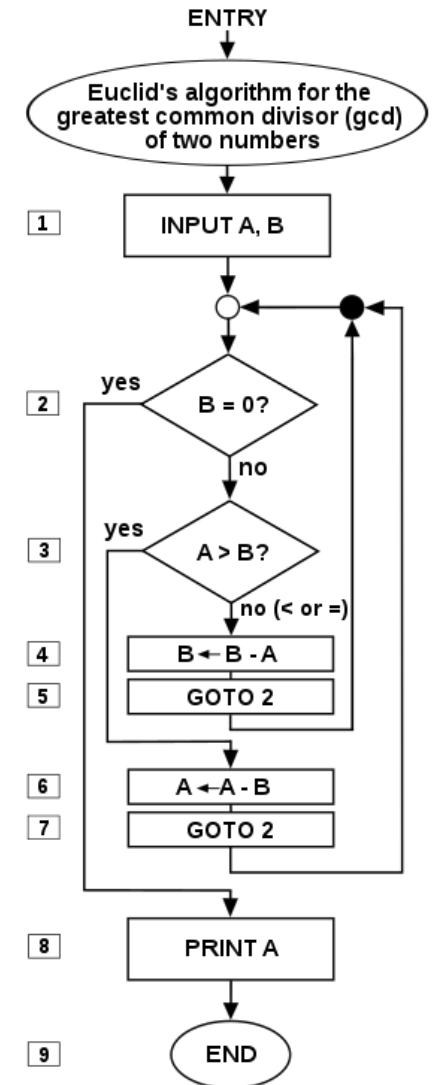
Learning objectives

After this lecture, you should be able to:

- Explain what an algorithm is
- Explain the big "O" notation as a measure of algorithm complexity and provide examples
- Name and describe simple algorithms for a simple task, e.g. sorting

What is an algorithm?

- A defined set of **steps** to solve a particular **problem** of interest.
- Algorithm are like road maps from your input data to your output, getting there in a reproducible fashion.
- An algorithm has these characteristics
 - Explicit, complete, and precise **initial** conditions
 - A finite, complete, unbroken series of **steps** to follow to arrive at the desired result
 - Explicit, complete and precise **terminal** (stopping) conditions)
- There are two forms of algorithms
 - Flow chart
 - Pseudocode (see Lecture 4.1)



How could we evaluate the efficiency of our programmes?

- Measure with a **timer**
- **Count** the operations
- Abstract notion of **order of growth**
 - Assess the impact of choices of algorithm in solving a problem
 - Measure the inherent difficulty in solving a problem.

Search Example

- Lets consider a function that searches for an element (e) in a list (L)

```
def search_for_elt(L, e):  
    for i in L:  
        if i == e:  
            return True  
    return False
```

When e is **first element** in the list → best case

When e is **not in the list** → worst case

When you have to **look through about half** of elements in the list → average case.

Want to measure this behaviour in a general way.

Best, average, worst cases

- Suppose you are given a list L of some length $\text{len}(L)$
- **Best case**: minimum running time over all possible inputs of a given size, $\text{len}(L)$
 - Constant for `search_for_elmnt`
 - First element in any list

Best, average, worst cases

- Suppose you are given a list L of some length $\text{len}(L)$
- **Best case**: minimum running time over all possible inputs of a given size, $\text{len}(L)$
 - Constant for `search_for_elemnt`
 - First element in any list
- **Average case**: average running time over all possible inputs of a given size, $\text{len}(L)$
 - Practical measure

Best, average, worst cases

- Suppose you are given a list L of some length $\text{len}(L)$
- **Best case**: minimum running time over all possible inputs of a given size, $\text{len}(L)$
 - Constant for `search_for_elemnt`
 - First element in any list
- **Average case**: average running time over all possible inputs of a given size, $\text{len}(L)$
 - Practical measure
- **Worst case**: maximum running time over all possible inputs of a given size, $\text{len}(L)$
 - Linear in length of list for `search_for_elemnt`
 - Must search entire list and not find it

Orders of growth

- Want to evaluate a program's efficiency when the **input is very big**.
 - Want to express the **growth of program's run time** as input size grows.
 - Want to put an **upper bound** on growth – as tight as possible.
 - Do not need to be precise: **"order of" not "exact"** growth.
 - We will look at the **largest factors** in run time (which section of the program will take the longest to run?)
-
- Generally, we want a tight upper bound on growth as a function of the size of input → the worst case scenario.

Measuring order of growth: Big "O" notation

- The Big "O" notation measures this **upper bound on the asymptotic growth**, often called the order of growth.
- **Big "O" or $O()$** is used to describe worse case scenarios.
 - This occurs often(!) and is the bottleneck when a program runs
 - Used to express rate of growth of program relative to the input size
 - Evaluates the algorithm **not** the CPU or implementation

What does $O(N)$ measure?

- We are interested in describing how the amount of time needed grows as size of (or input to) a problem grows.
- Thus, given an expression for the number of operations needed to compute an algorithm, we want to know the asymptotic behaviour as the size of the problem gets large.
- Hence we focus on terms that grow most rapidly.

Some simplification examples

- Drop constants and multiplicative factors
- Focus on **dominant terms**
- $O(n^2) = n^2 + 2n + 2$

Some simplification examples

- Drop constants and multiplicative factors
- Focus on **dominant terms**
- $O(n^2) = n^2 + \text{X}1 + \text{X}$

Some simplification examples

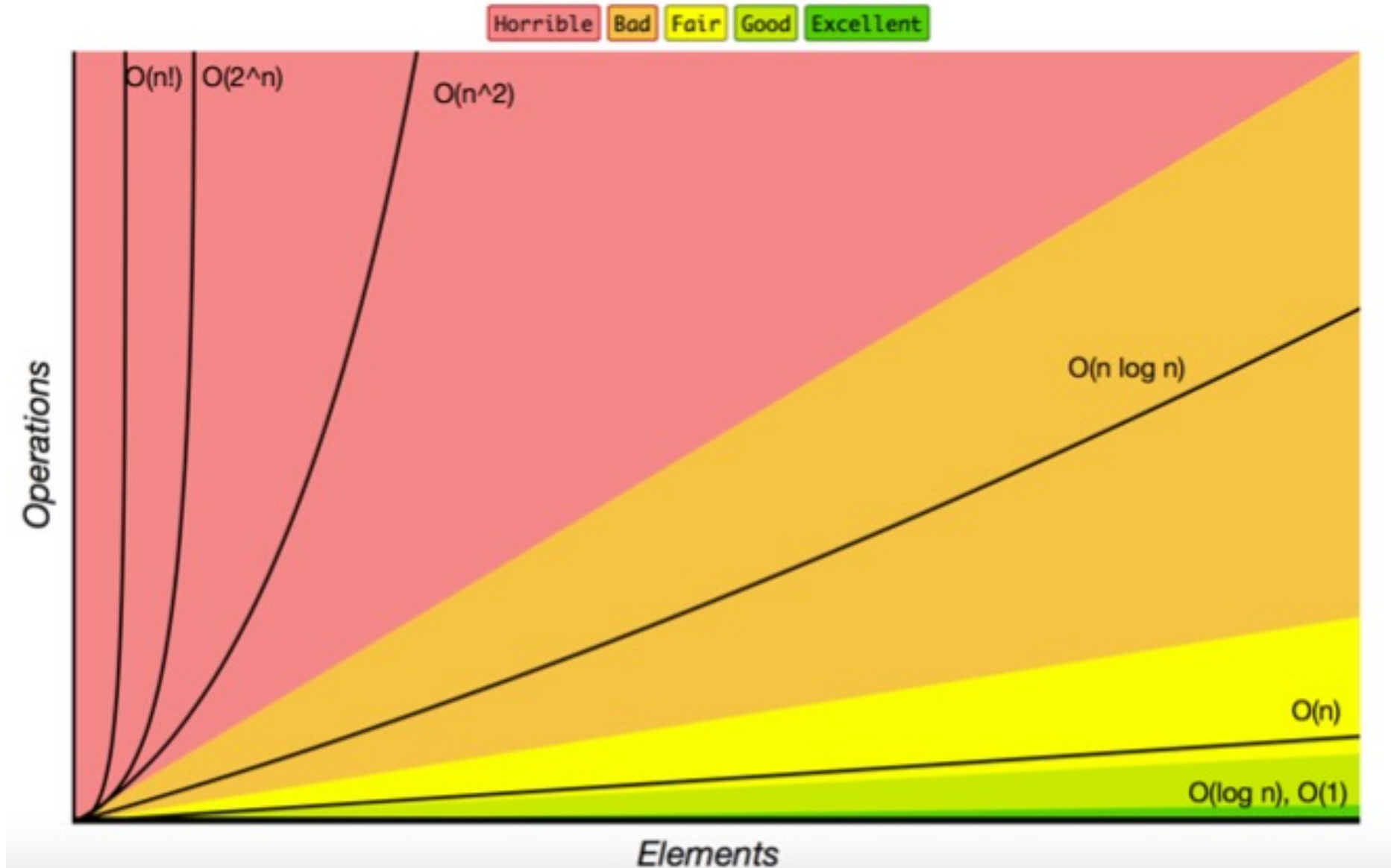
- Drop constants and multiplicative factors
- Focus on **dominant terms**
- $O(n^2) = n^2 + 2n + 2$
- $O(n^2) = n^2 + 100000n + 3^{1000}$

Some simplification examples

- Drop constants and multiplicative factors
- Focus on **dominant terms**

- $O(n^2) = n^2 + 2n + 2$
- $O(n^2) = n^2 + 1000000n + 3^{1000}$
- $O(n) = \log(n) + n + 3$
- $O(n \log n) = 0.001 * n * \log(n) + 300n$
- $O(3^n) \quad 2n^{30} + 3^n$

The Big "O" complexity chart



How to analyse program complexity

- **Combine** complexity classes
- Analyse statements inside functions
- Apply some results, focus on the dominant term
- **Law of Addition** for $O()$:
 - Used with **sequential** statements, e.g. $O(f(n)) + O(g(n))$ is $O(f(n) + g(n))$

How to analyse program complexity

- **Combine** complexity classes
- Analyse statements inside functions
- Apply some results, focus on the dominant term
- **Law of Addition** for $O()$:
 - Used with **sequential** statements, e.g. $O(f(n)) + O(g(n))$ is $O(f(n) + g(n))$
 - For example

```
for i in range(n):  
    print('a')  
for j in range(n*n):  
    print('b')
```

A red curly brace groups the two complexity classes $O(n)$ and $O(n^2)$ on the left, with the result $O(n) + O(n^2)$ on the right.

is $O(n) + O(n*n) = O(n+n^2) = O(n^2)$ because of the dominant term

How to analyse program complexity

- **Combine** complexity classes
- Analyse statements inside functions
- Apply some results, focus on the dominant term
- **Law of Addition** for $O()$:
- Used with **sequential** statements, e.g. $O(f(n)) + O(g(n))$ is $O(f(n) + g(n))$
- For example

```
for i in range(n):  
    for j in range(n):  
        print('a')
```

$O(n)$ } n loops, each $O(n)$
 $O(n) * O(n)$

is $O(n) * O(n) = O(n * n) = O(n^2)$ because the outer loops goes n times and the inner loop goes n times for ever outer loop iteration

Sort algorithms

- Sorting is a process that organises a collection of data into either ascending or descending order
- There are many sorting algorithms, such as:
 - Bubble sort
 - Merge sort
 - etc, etc, etc
- Which one is more efficient?

Bubble sort

- **Compares consecutive pairs** of elements
- Swap elements in the pair such that the smallest is first
- When you reach the end of the list, **start over** again
- Stop when **no more swaps** have been made
- The largest unsorted element is always at the end after the first pass, so at most n passes



How complex is bubble sort?

```
def bubble_sort(L):
    swap = False
    while not swap:
        swap = True
        for j in range(1, len(L)):
            if L[j-1] > L[j]:
                swap = False
                temp = L[j]
                L[j] = L[j-1]
                L[j-1] = temp
```



How complex is bubble sort?

```
def bubble_sort(L):  
    swap = False  
    while not swap:  
        swap = True  
        for j in range(1, len(L)):  
            if L[j-1] > L[j]:  
                swap = False  
                temp = L[j]  
                L[j] = L[j-1]  
                L[j-1] = temp
```

O(len(L))

- The inner for loop is doing the **comparisons**

How complex is bubble sort?

```
def bubble_sort(L):  
    swap = False  
    while not swap: O(len(L))  
        swap = True  
        for j in range(1, len(L)): O(len(L))  
            if L[j-1] > L[j]:  
                swap = False  
                temp = L[j]  
                L[j] = L[j-1]  
                L[j-1] = temp
```

- The inner for loop is doing the **comparisons**
- The outer while loop is doing **multiple passes** until no more swaps

How complex is bubble sort?

```
def bubble_sort(L):  
    swap = False  
    while not swap:  
        swap = True  
        for j in range(1, len(L)):  
            if L[j-1] > L[j]:  
                swap = False  
                temp = L[j]  
                L[j] = L[j-1]  
                L[j-1] = temp
```

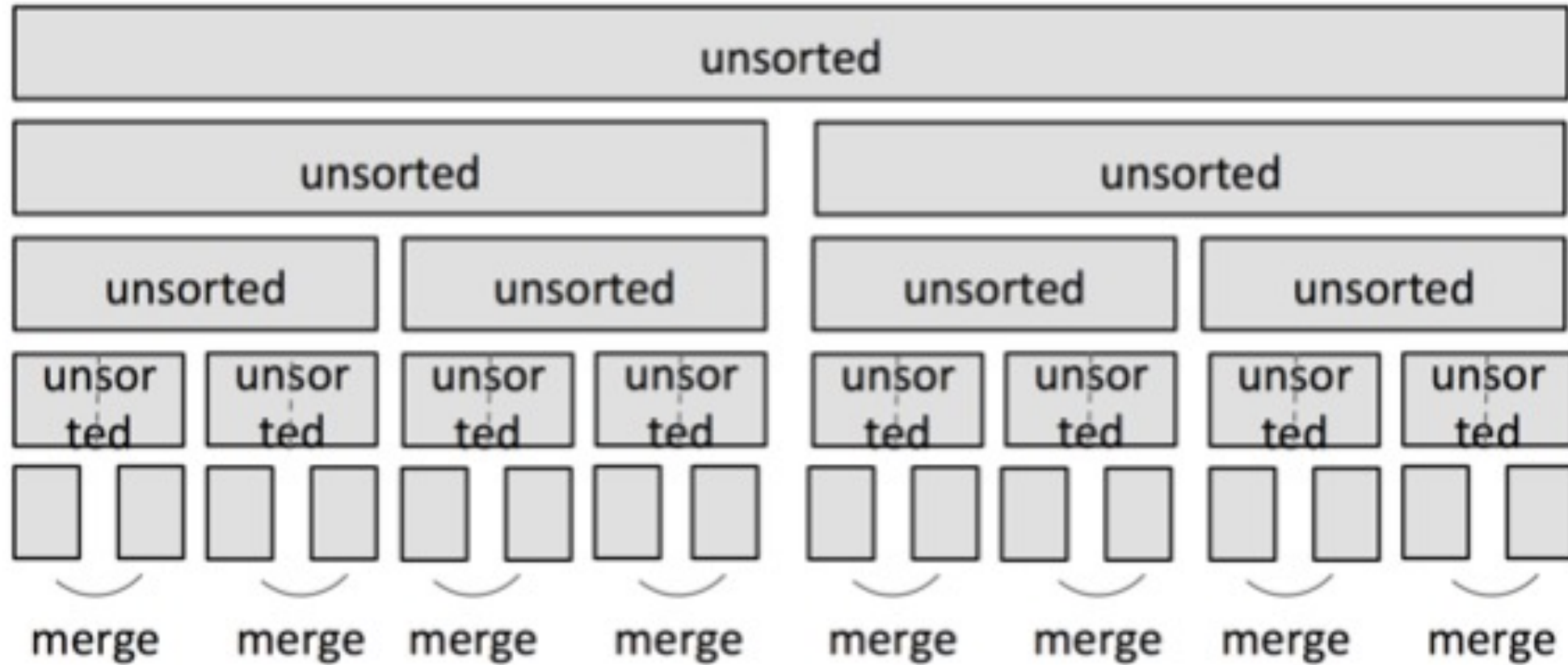
- The inner for loop is doing the **comparisons**
- The outer while loop is doing **multiple passes** until no more swaps
- **$O(n^2)$ where n is $\text{len}(L)$** to do $\text{len}(L)-1$ comparisons and $\text{len}(L)-1$ passes

Merge sort

- Use a divide-and conquer approach:
 1. If a list is of length 0 or 1, it must already be sorted
 2. If a list has more than one element, split into two lists and sort each
 3. Merge sorted sublists
 - Look at first element of each, move smaller to the end of the result
 - When one list is empty, just copy the rest of the other list

Merge sort

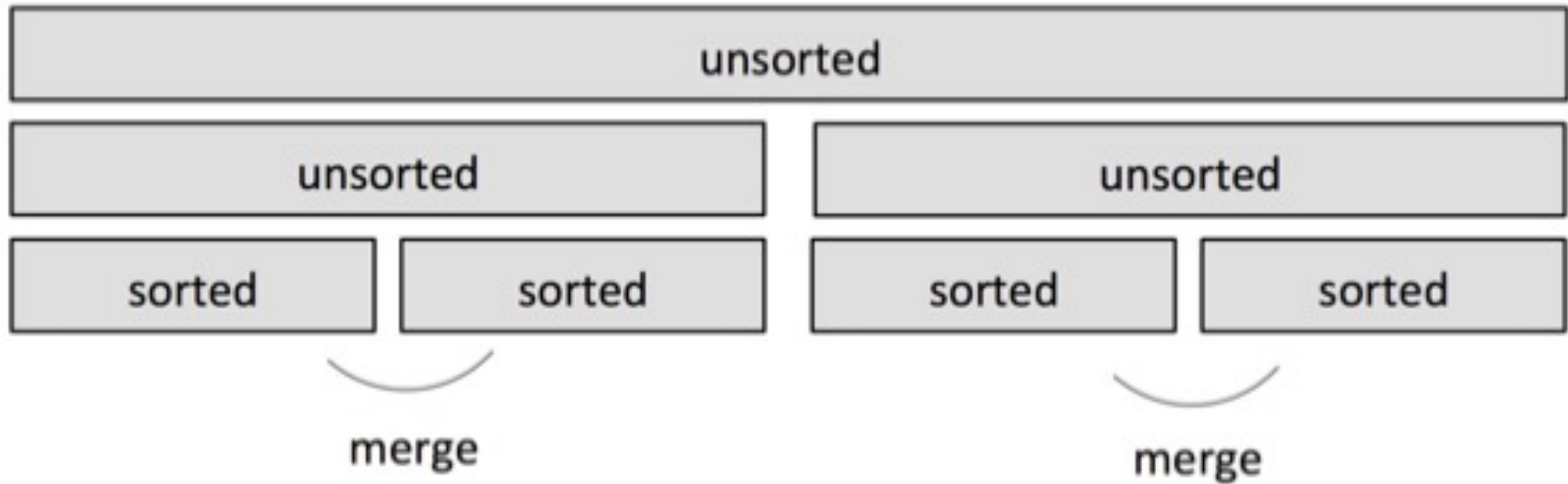
- Use a divide-and conquer approach



- Split **list in half** until you have sublists of only 1 element.

Merge sort

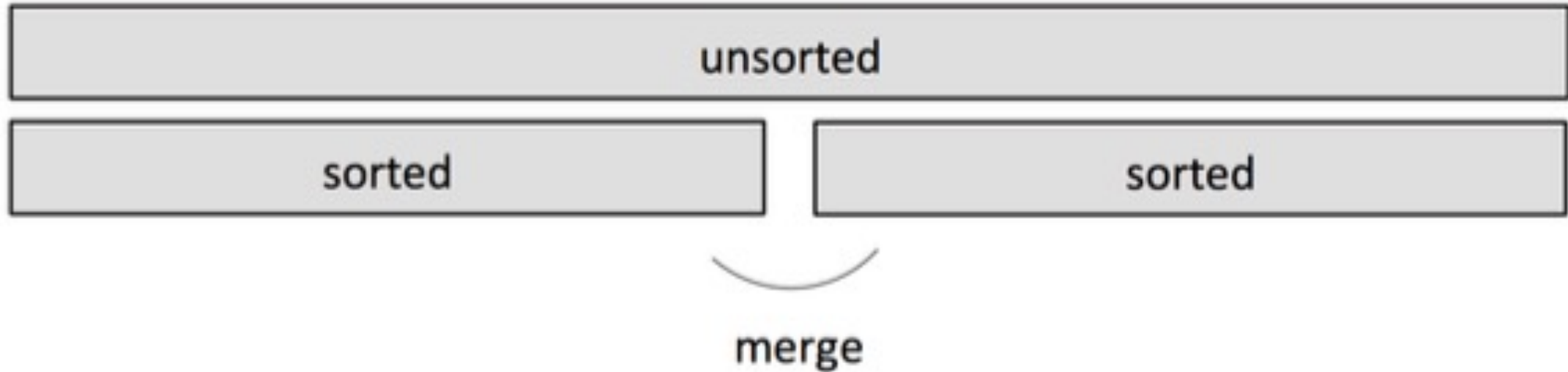
- Use a divide-and conquer approach



- Merge sorted sublists
- Sublists will be sorted after merge.

Merge sort

- Use a divide-and conquer approach



- Merge sorted sublists
- Sublists will be sorted after merge.

Merge sort

- Use a divide-and conquer approach – done!



sorted

Example of merging

Left in list 1

[1]5,12,18,19,20]

[5]12,18,19,20]

Left in list 2

[2]3,4,17]

[2]3,4,17]

Compare

1, 2 → []

5, 2 → [1]

Result

Example of merging

Left in list 1

[1, 5, 12, 18, 19, 20]

[5, 12, 18, 19, 20]

[5, 12, 18, 19, 20]

Left in list 2

[2, 3, 4, 17]

[2, 3, 4, 17]

[3, 4, 17]

Compare

1, 2 → [1]

5, 2 → [1, 2]

5, 3 → [1, 2, 3]

Result

Example of merging

Left in list 1	Left in list 2	Compare	Result
[1,5,12,18,19,20]	[2,3,4,17]	1, 2 →	[]
[5,12,18,19,20]	[2,3,4,17]	5, 2 →	[1]
[5,12,18,19,20]	[3,4,17]	5, 3 →	[1, 2]
[5,12,18,19,20]	[4,17]	5, 4	[1, 2, 3]
[5,12,18,19,20]	[17]	5, 17	[1, 2, 3, 4]
[12,18,19,20]	[17]	12, 17	[1, 2, 3, 4, 5]
[18,19,20]	[17]	18, 17	[1, 2, 3, 4, 5, 12]
[18,19,20]	[]	18, --	[1, 2, 3, 4, 5, 12, 17]
[]	[]		[1, 2, 3, 4, 5, 12, 17, 18, 19, 20]

Merging sublists step

```
def merge(left, right):  
    result = []  
    i, j = 0, 0  
    while i < len(left) and j < len(right):  
        if left[i] < right[j]:  
            result.append(left[i])  
            i += 1  
        else:  
            result.append(right[j])  
            j += 1  
    while (i < len(left)):  
        result.append(left[i])  
        i += 1  
    while (j < len(right)):  
        result.append(right[j])  
        j += 1  
    return result
```

Merging sublists step

```
def merge(left, right):  
    result = []  
    i, j = 0, 0  
    while i < len(left) and j < len(right):  
        if left[i] < right[j]:  
            result.append(left[i])  
            i += 1  
        else:  
            result.append(right[j])  
            j += 1  
    while i < len(left):  
        result.append(left[i])  
        i += 1  
    while j < len(right):  
        result.append(right[j])  
        j += 1  
    return result
```

- Left and right sublists are ordered
- Move indices for sublists depending on which sublist holds next smaller element

Merging sublists step

```
def merge(left, right):  
    result = []  
    i, j = 0, 0  
    while i < len(left) and j < len(right):  
        if left[i] < right[j]:  
            result.append(left[i])  
            i += 1  
        else:  
            result.append(right[j])  
            j += 1  
    while (i < len(left)):  
        result.append(left[i])  
        i += 1  
    while (j < len(right)):  
        result.append(right[j])  
        j += 1  
    return result
```

- Left and right sublists are ordered
- Move indices for sublists depending on which sublist holds next smaller element

- When right sublist is empty

Merging sublists step

```
def merge(left, right):  
    result = []  
    i, j = 0, 0  
    while i < len(left) and j < len(right):  
        if left[i] < right[j]:  
            result.append(left[i])  
            i += 1  
        else:  
            result.append(right[j])  
            j += 1  
    while (i < len(left)):  
        result.append(left[i])  
        i += 1  
    while (j < len(right)):  
        result.append(right[j])  
        j += 1  
    return result
```

- Left and right sublists are ordered
- Move indices for sublists depending on which sublist holds next smaller element

- When right sublist is empty

- When left sublist is empty

Complexity of merging sublists step

- You go through two lists, but only one pass
- You compare only the **smallest elements in each sublist**
- $O(\text{len}(\text{left}) + \text{len}(\text{right}))$ copied elements
- $O(\text{len}(\text{longer list}))$ comparisons
- **Linear increase with the length of list**

Merge sort algorithm – the recursive stage

```
def merge_sort(L):  
    if len(L) < 2:  
        return L  
    else:  
        middle = len(L)//2  
        left = merge_sort(L[:middle])  
        right = merge_sort(L[middle:])  
        return merge(left, right)
```

- The list is **divided** successively into halves.
- This depth-first approach **conquers the smallest pieces down one branch** first before moving to larger pieces.

Merge sort algorithm – the recursive stage

```
def merge_sort(L):  
    if len(L) < 2:  
        return L  
    else:  
        middle = len(L)//2  
        left = merge_sort(L[:middle])  
        right = merge_sort(L[middle:])  
        return merge(left, right)
```

base case

- The list is **divided** successively into halves.
- This depth-first approach **conquers the smallest pieces down one branch** first before moving to larger pieces.

Merge sort algorithm – the recursive stage

```
def merge_sort(L):  
    if len(L) < 2:  
        return L[:] base case  
    else:  
        middle = len(L)//2  
        left = merge_sort(L[:middle])  
        right = merge_sort(L[middle:]) divide  
        return merge(left, right)
```

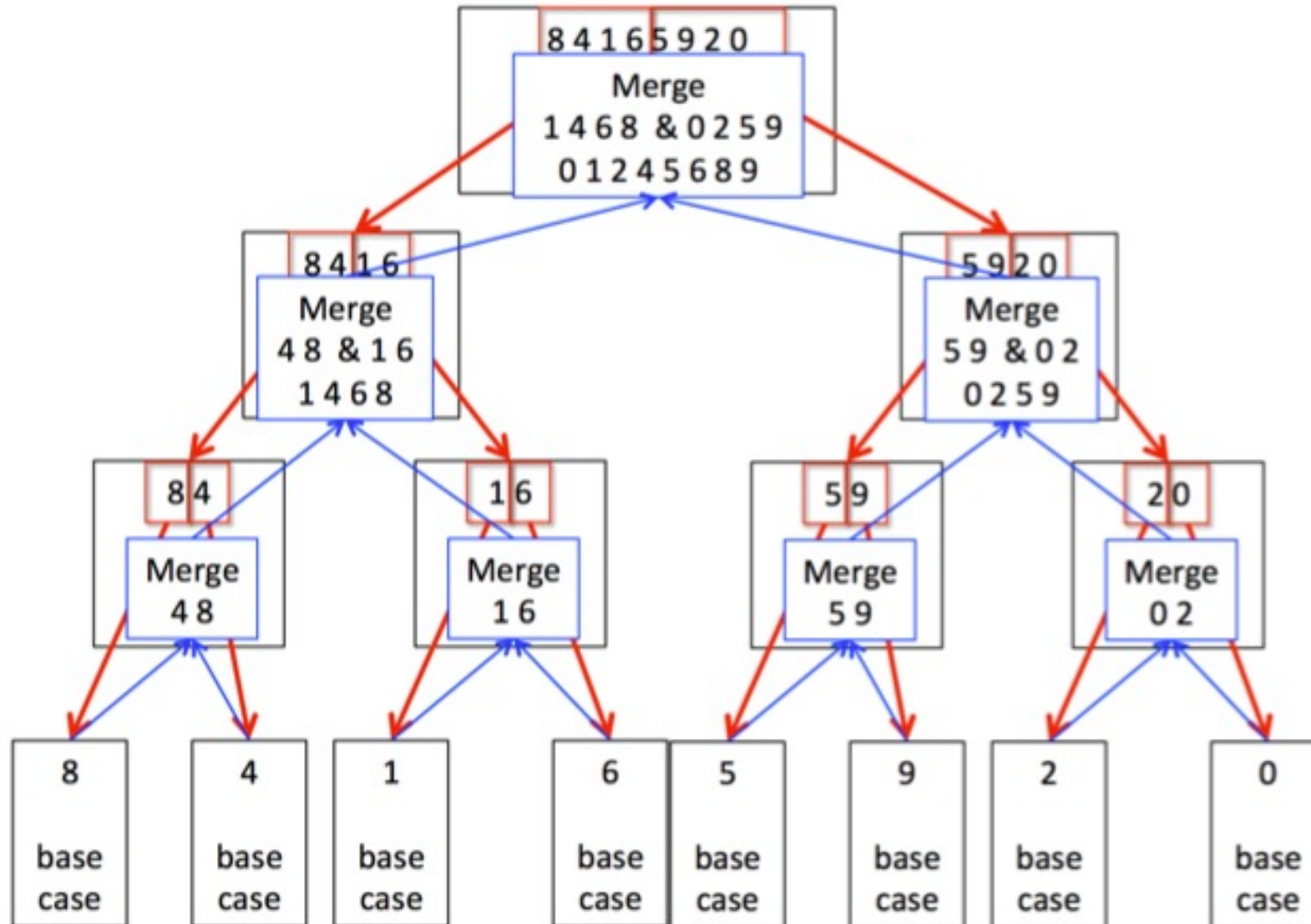
- The list is **divided** successively into halves.
- This depth-first approach **conquers the smallest pieces down one branch** first before moving to larger pieces.

Merge sort algorithm – the recursive stage

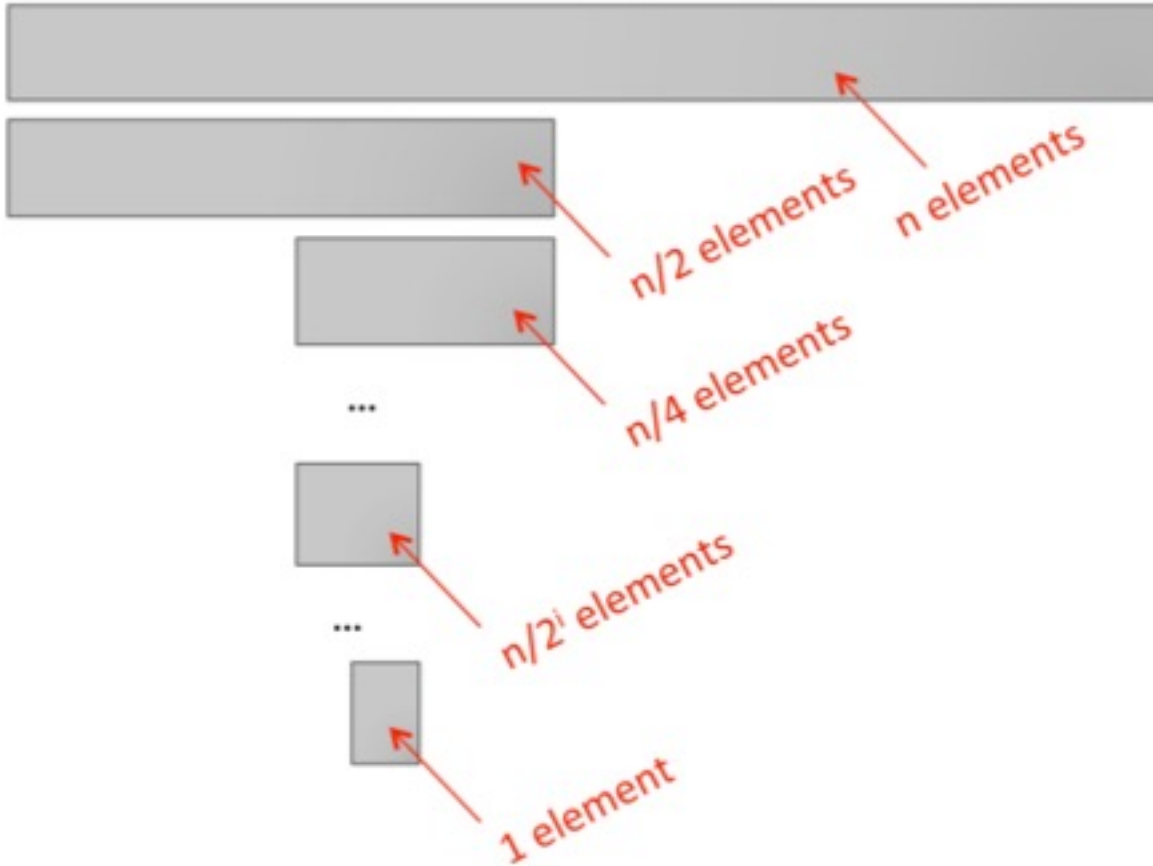
```
def merge_sort(L):  
    if len(L) < 2:  
        return L[:] base case  
    else:  
        middle = len(L)//2  
        left = merge_sort(L[:middle])  
        right = merge_sort(L[middle:]) divide  
        return merge(left, right) conquer with the merge step
```

- The list is **divided** successively into halves.
- This depth-first approach **conquers the smallest pieces down one branch** first before moving to larger pieces.

Merge sort algorithm – the recursive stage



The complexity of recursion



- Finish looking through the list when $1 = n/2^i$ so $i = \log(n)$
- The complexity of recursion is **$O(\log(n))$** where n is $\text{len}(L)$.

How complex is merge sort?

- At the **first recursion level**:
 - $n/2$ elements in each list
 - $O(n) + O(n) = O(n)$ where n is $\text{len}(L)$
- At the **second recursion level**:
 - $n/4$ elements in each list
 - two merges $\rightarrow O(n)$ where n is $\text{len}(L)$
- Each recursion level is $O(n)$ where n is $\text{len}(L)$
- **Dividing the list in half** with each recursive call
 - $O(\log(n))$ where n is $\text{len}(L)$
- Overall complexity is **$O(n \log(n))$ where n is $\text{len}(L)$.**

Summary

- $O(1)$ – code does not depend on the size of each problem.
- $O(\log n)$ – you reduce the problem in half each time through the process.
- $O(n)$ – simple iterative or recursive programs.
- $O(n \log(n))$ – merge sort, bisection search.
- $O(n^c)$ – nested loops or recursive calls.
- $O(c^n)$ – multiple recursive calls at each level.

$O(n^c)$ & $O(c^n)$, while c is a constant, say “2”:

$O(n^2)$ example : double cycle

for i in range(n): # Outer cycle n times

 for j in range(n): # Inner cycle n times

 # ... Excute constant computation functions...

$O(2^n)$ example : Naive recursion of the Fibonacci sequence (extremely inefficient)

def fib(n):

 if n <= 1:

 return n

 return fib(n-1) + fib(n-2) # Each level generates two new recursive calls.

Learning objectives

Now, you should be able to:

- Explain what an algorithm is
- Explain the big "O" notation as a measure of algorithm complexity and provide examples
- Name and describe simple algorithms for a simple task, e.g. sorting



浙江大学爱丁堡大学联合学院

ZJU-UoE Institute

Python IV: Functions and Algorithms

Algorithm Complexity

Any questions?

IBI 1, Lecture 8.2

Dr. Ying CHI – yingchi@intl.zju.edu.cn

Adapted from Dr Rob Young

Semester 2, 2025/26